

PROJECT DOCUMENTATION

Automated Detection and Grading of Diabetic Retinopathy Using CNN

A deep-learning pipeline that grades retinal fundus photographs on the international 0–4 severity scale

Dataset APTOS 2019 Blindness Detection (3,662 fundus images)

Best model EfficientNet-B3 — QWK 0.9005, accuracy 82.5%

Compiled 09 September 2026

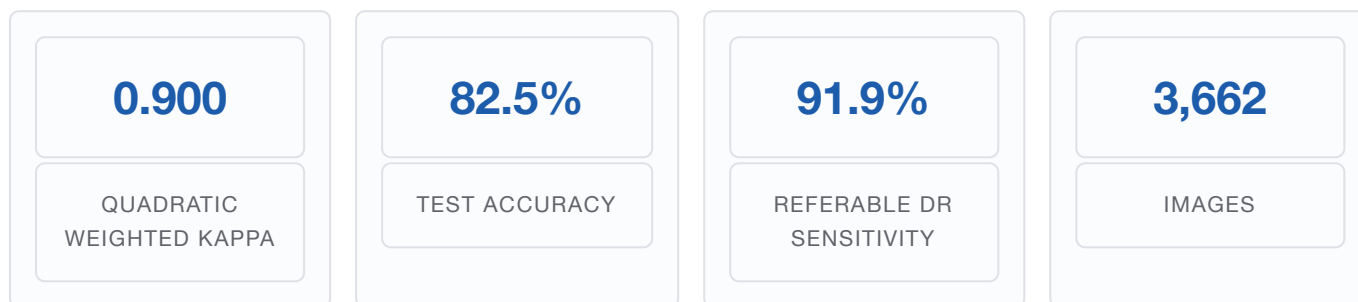
Contents

1. Project overview
2. The clinical problem and grading scale
3. Dataset
4. Flow of implementation — the seven stages
5. Technology stack
6. Algorithms and techniques
7. Preprocessing in detail
8. Model architectures
9. Training configuration
10. Evaluation metrics
11. Results
12. Explainability
13. The demo application
14. Repository structure
15. How to run it
16. Known limitations and future work
17. Glossary

1 Project overview

Diabetic retinopathy is damage to the retina caused by diabetes and is a leading cause of preventable blindness. It is diagnosed by an ophthalmologist examining a photograph of the back of the eye — a **fundus image** — and assigning a severity grade. Screening programmes generate far more images than there are specialists to read them.

This project trains a convolutional neural network to assign that grade automatically, and to explain each decision rather than emitting a bare number.



Objectives

1. Grade a fundus photograph 0–4 on the international DR severity scale.
2. Quantify what transfer learning is worth, by comparing a from-scratch CNN against a pretrained backbone under identical conditions.
3. Explain each prediction — both *where* the network looked and *what* is in that region.
4. Run entirely on local hardware, with no dependency on cloud GPUs.

Scope. This is a research prototype built for academic assessment. It is not a medical device and has not been clinically validated. The title refers to grading in patients with Type 1 diabetes; the dataset records DR *severity* and carries no diabetes-type field, so the model predicts severity only.

2 The clinical problem and grading scale

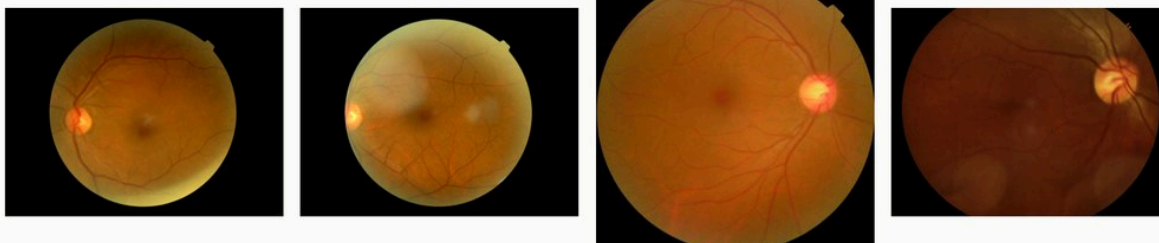
Grades follow the International Clinical Diabetic Retinopathy scale. They are **ordinal**: the classes have a meaningful order, and an error between distant grades is far worse than one between neighbours.

Grade	Name	Images	Share	Clinical action
0	No DR	1,805	49.3%	Routine annual screening
1	Mild	370	10.1%	Re-screen in 12 months
2	Moderate	999	27.3%	Referable — ophthalmologist review
3	Severe	193	5.3%	Referable — prompt referral
4	Proliferative DR	295	8.1%	Urgent referral

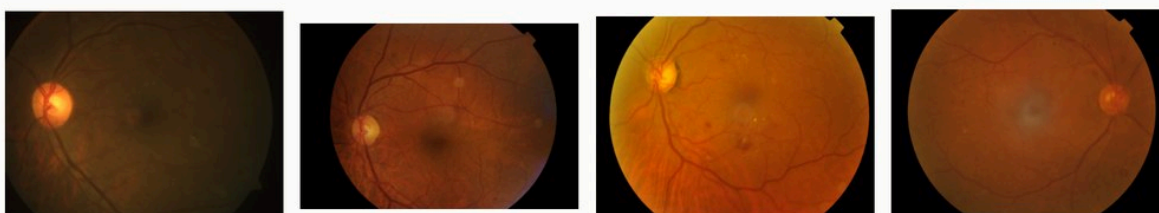
Referable DR. Grades 2 and above are "referable" — the threshold at which a screening programme sends a patient to a specialist. Collapsing five grades into this yes/no decision gives the number a clinical reader cares about most, because missing a severe case costs eyesight while a false alarm costs one appointment. Reported separately throughout.

APTOS 2019 - sample fundus images by DR grade

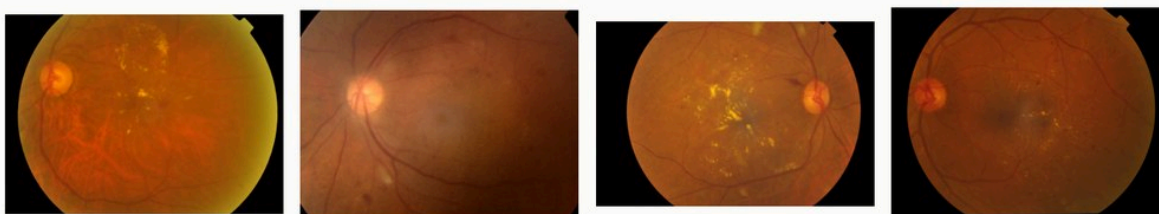
Grade 0
No DR



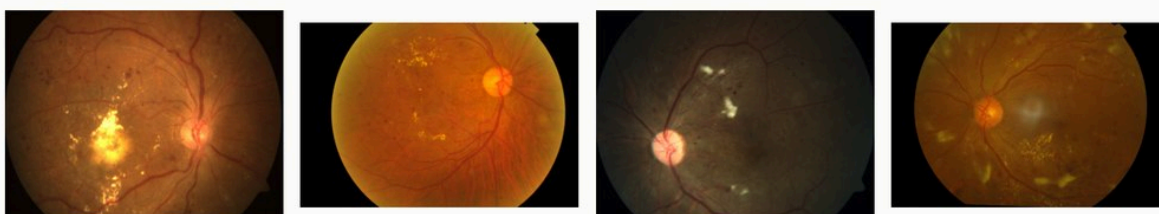
Grade 1
Mild



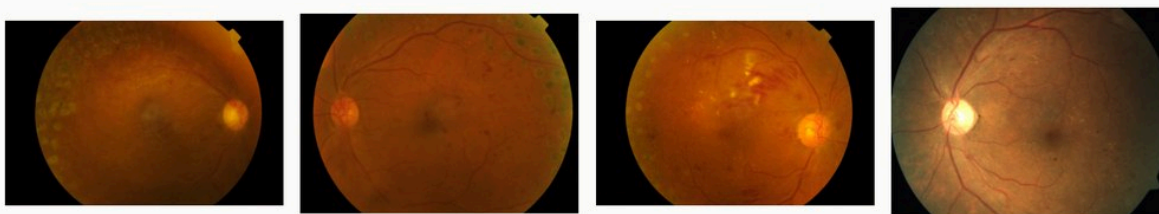
Grade 2
Moderate



Grade 3
Severe



Grade 4
Proliferative



Representative fundus images at each grade. The difference between grade 0 and grade 4 is obvious; adjacent grades are subtle, and that is where most errors occur.

3 Dataset

Source

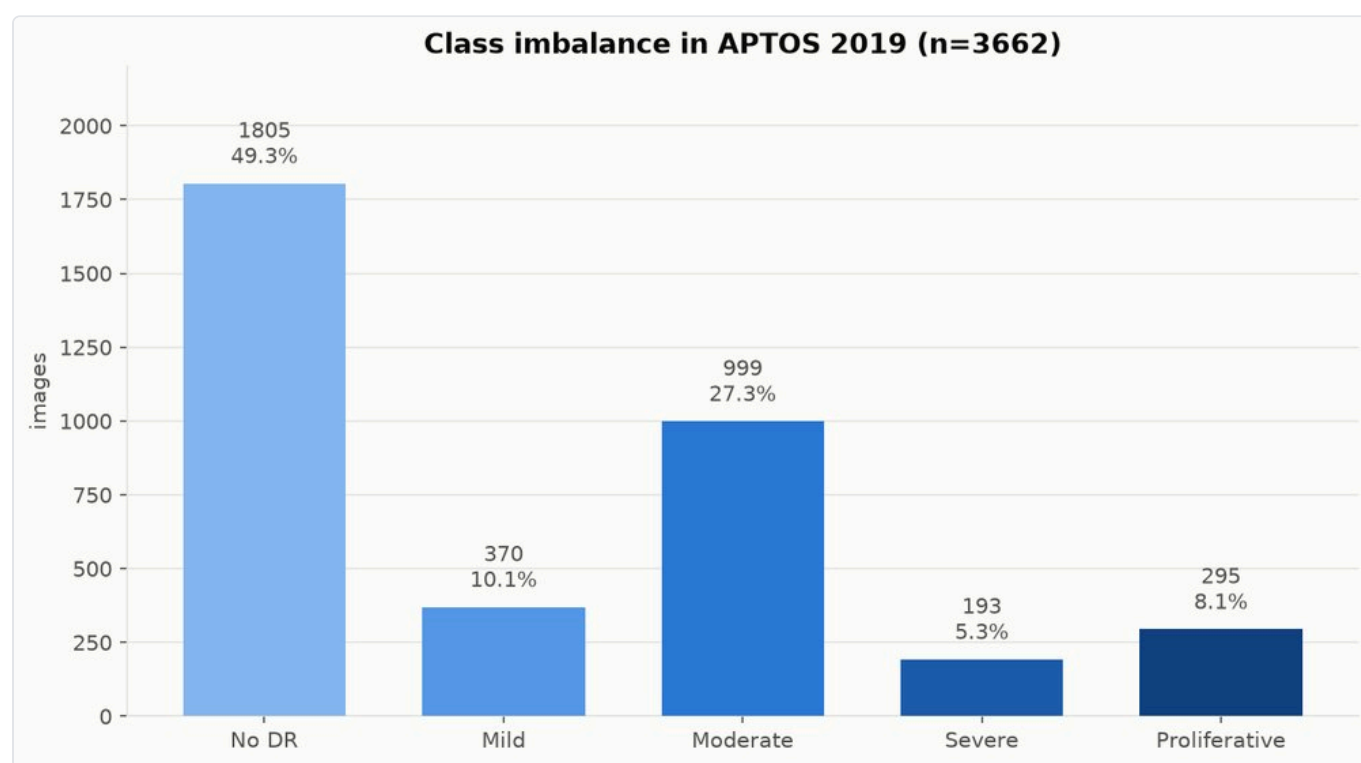
APTOS 2019 Blindness Detection, collected by Aravind Eye Hospital in rural India and released as a Kaggle competition. 3,662 colour fundus photographs, each graded 0–4 by a clinician.

<https://www.kaggle.com/competitions/aptos2019-blindness-detection>

Access note. The competition blocks every download, even a 28 KB file, until its rules are accepted in a browser. The download script therefore defaults to a complete mirror of the same images ([mariaherrerot/aptos2019](#)) needing only a logged-in CLI, and verifies the grade counts against the official distribution either way.

Class distribution

The dataset is heavily imbalanced, which drives several design decisions.



Grade distribution. Nearly half of all images are grade 0.

Splitting

Split	Images	Share	Purpose
Train	2,562	70%	Weights are learned from these
Val	550	15%	Chooses the best epoch and triggers early stopping
Test	550	15%	Touched once, at the very end

The split is **stratified**, so each grade keeps its proportion in all three sets. With only 193 grade-3 images in total, a random split could leave wildly different balances and make the validation curve meaningless.

Data quality findings

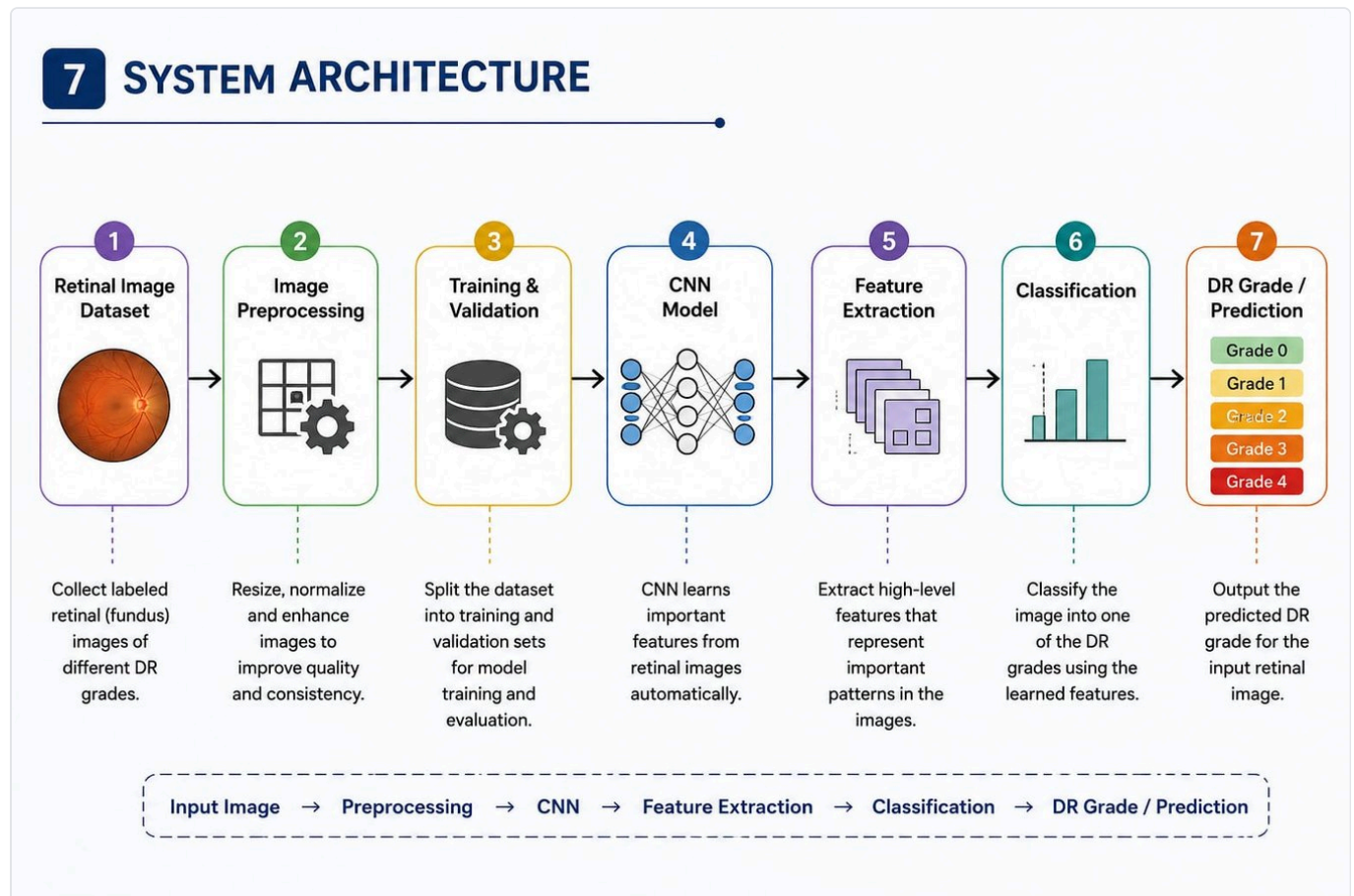
Hashing every image surfaced two problems in the dataset itself.

- **Duplicate photographs.** 3,662 rows contain only 3,534 unique images — 128 rows are pixel-identical copies under a different `id_code`, across 123 groups.
- **Conflicting labels.** 30 of those groups are graded *differently* in different rows — irreducible label noise that puts a ceiling on achievable accuracy.

Under a naive per-row split, 28 of 550 test images (5.1%) had a twin in train. Measuring the effect showed the leakage did **not** inflate the result: removing those rows moves QWK from 0.9005 to 0.9031, slightly *better*, because the duplicated rows are the ones carrying conflicting labels. Splitting is now done on image-content hash so copies cannot span splits.

4 Flow of implementation

The pipeline runs in seven stages. Each is a separate module, so any stage can be re-run without repeating the others.



System architecture: the seven stages from raw image to predicted grade.

#	Stage	Module	What happens
1	Retinal image dataset	<code>src/download_data.py</code>	Downloads and verifies 3,662 graded fundus images.
2	Image preprocessing	<code>src/preprocess.py</code>	Crop, resize, contrast enhancement, circular mask. Cached to disk.
3	Training / validation split	<code>src/preprocess.py</code>	Stratified, group-aware 70/15/15 split.
4	CNN model	<code>src/models.py</code>	Baseline CNN, EfficientNet-B3 or ResNet-50.
5	Feature extraction	<code>src/models.py</code>	The backbone converts an image into a feature vector.
6	Classification	<code>src/models.py</code> , <code>src/engine.py</code>	A linear head maps features to five grade scores.
7	DR grade / prediction	<code>src/predict.py</code> , <code>app/app.py</code>	Grade, confidence, referable decision, and explanation.

End-to-end sequence

```

Raw fundus image (e.g. 3216 x 2136)
|
|-- 1. Load and verify against the official grade distribution
|
|-- 2. Crop black border -> resize 456x456 -> Ben Graham enhancement
|           -> circular field-of-view mask
|           (cached as PNG so training never re-decodes)
|
|-- 3. Stratified group-aware split: 2,562 / 550 / 550
|
|-- 4. Augment (flip, rotate, scale, brightness, dropout)
|       -> resize to model input (300px) -> normalise (ImageNet stats)
|
|-- 5. Backbone -> 1,536-dimensional feature vector
|
|-- 6. Dropout -> Linear(1536 -> 5) -> softmax
|
|-- 7. Predicted grade + confidence + Grad-CAM + lesion analysis

```

5 Technology stack

Layer	Component	Role
Language	Python 3.14.7	Runtime for the whole pipeline
Deep learning	PyTorch 2.14.0	Model definition, autograd, training loop
	torchvision 0.29.0	Vision utilities
	timm 1.0.29	Pretrained EfficientNet-B3 and ResNet-50 backbones
Acceleration	Apple Metal (MPS)	GPU backend; falls back to CUDA, then CPU
Image processing	OpenCV 5.0.0	Preprocessing, lesion detection, Grad-CAM overlay
	Pillow 12.3.0	Image I/O
Augmentation	albumentations 2.0.8	Training-time augmentation pipeline
Data	pandas 3.0.5	Labels, splits, metric tables
	NumPy 2.5.3	Array operations
Metrics	scikit-learn 1.9.0	QWK, F1, confusion matrix, stratified splitting
Visualisation	matplotlib 3.11.1	All report figures
	seaborn 0.13.2	Confusion-matrix heatmaps
Application	Streamlit 1.63.0	Interactive demo
Dataset access	kaggle 2.2.4	Authenticated dataset download
Version control	Git + GitHub	History; weights shipped as release assets

Hardware

Development and training on an **Apple M5, 16 GB unified memory**, using PyTorch's Metal Performance Shaders backend with sustained GPU utilisation above 95%. `get_device()` selects MPS, then CUDA, then CPU, so the same code runs unchanged on an NVIDIA machine or CPU-only laptop.

Model	Input px	Batch	s/step	img/s	min/epoch	Total run
Baseline CNN	224	32	0.33 s	96.9	0.4 min	22 min
EfficientNet-B3	300	16	0.72 s	22.1	1.9 min	46 min
ResNet-50	300	16	0.55 s	29.1	1.5 min	~37 min

6 Algorithms and techniques

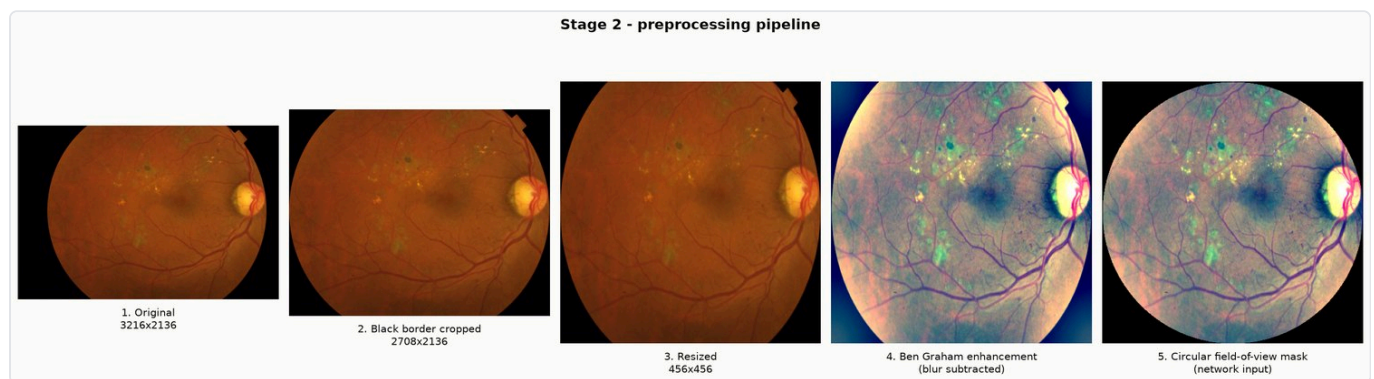
Algorithm	Role	What it does and why
Ben Graham preprocessing	Preprocessing	Subtracts a heavily Gaussian-blurred copy of the image from itself, removing the low-frequency component. Cancels per-camera colour cast and uneven illumination, leaving lesions visible.
Convolutional Neural Network	Model	Learns spatial features through stacked convolution, batch normalisation and non-linear activation.
Transfer learning	Model	ImageNet-pretrained backbone fine-tuned end to end. Reaches the from-scratch model's best score in 4 epochs instead of 32.
ReLU / SiLU	Activation	The non-linearity inside the network. Baseline and ResNet use ReLU; EfficientNet uses SiLU (Swish). Without one, stacked convolutions collapse into a single linear operation.
AdamW	Optimiser	Adam with decoupled weight decay. Adapts a learning rate per parameter, which converges reliably on a small dataset.
Cosine annealing + warmup	LR schedule	Two epochs of linear warmup, then cosine decay. Warmup stops the randomly initialised head washing out pretrained features.
Class-weighted cross-entropy	Loss	Inverse-frequency weights. Grade 0 is 49% of the data, so an unweighted model scores ~49% by always predicting 'No DR'.
Label smoothing (0.05)	Loss	Softens targets to reduce overconfidence.
Gradient clipping (norm 1.0)	Training	Caps gradient magnitude to prevent exploding updates.
Early stopping	Training	Halts after 8 epochs without validation QWK improvement, keeping the best checkpoint.
Stratified group splitting	Data	Splits on image-content hash so duplicate photographs cannot span train and test, while preserving each grade's proportion.
Quadratic Weighted Kappa	Metric	Ordinal agreement corrected for chance. Penalises a grade 0→4 error far more than 3→4.
Grad-CAM	Explainability	Backpropagates the predicted class score to the last convolutional feature map, weighting channels by mean gradient.
Morphological lesion detection	Explainability	Black top-hat for vessels, local-minimum analysis for red lesions, colour thresholding for exudates, with per-blob shape filtering.

A distinction worth being clear on. An **activation function** (ReLU, SiLU) is the non-linearity *inside* the network. An **optimiser** (AdamW, SGD) is the rule that turns gradients *into* weight updates. They do different jobs and every run uses one of each — they are not alternatives to one another.

7 Preprocessing in detail

Fundus images arrive at different resolutions, aspect ratios and exposures because they come from several camera models and clinics. Four operations normalise them.

1. **Black border crop.** Trims the uninformative frame around the circular retina.
2. **Square resize to 456 px**, cached to disk so training never re-decodes multi-megabyte originals.
3. **Ben Graham enhancement.** Subtracts a heavily Gaussian-blurred copy of the image from itself. Removing the low-frequency component cancels per-camera colour cast and uneven illumination, leaving microaneurysms, haemorrhages and exudates clearly visible. This is the single highest-impact step for this dataset.
4. **Circular field-of-view mask.** Zeroes the corners outside the retinal circle so the network cannot key on camera-specific artefacts.



The stage-2 pipeline applied to one image. Panel 4 shows the Ben Graham step bringing out exudates and vessel detail that are nearly invisible in the original.

Augmentation

Applied to the training split only. Retinal images have no canonical orientation — left and right eyes are mirror images — so flips and full rotations are safe and effectively multiply a small dataset.

Transform	Setting	Why
RandomResizedCrop	scale 0.85–1.0	Slight framing variation
Horizontal / vertical flip	p = 0.5 each	Eye laterality and camera orientation
Affine	$\pm 180^\circ$ rotation, $\pm 10\%$ scale, $\pm 5\%$ shift	No canonical orientation exists
RandomBrightnessContrast	$\pm 15\%$	Exposure differences between clinics
HueSaturationValue	small shifts	Camera colour variation
CoarseDropout	1–4 holes	Forces reliance on more than one region
Normalize	ImageNet mean/std	Matches what the pretrained backbone expects

8 Model architectures

Key	Architecture	Parameters	Input	Purpose
baseline	5-block VGG-style CNN, from scratch	4.85 M	224 px	No pretrained weights. Establishes the honest reference point.
efficientnet	EfficientNet-B3, ImageNet-pretrained	10.70 M	300 px	Main model. Compound-scaled depth, width and resolution.
resnet	ResNet-50, ImageNet-pretrained	23.52 M	300 px	Residual connections. Configured as a third comparison row.

Baseline CNN

Five convolutional blocks, each `Conv → BatchNorm → ReLU → Conv → BatchNorm → ReLU → MaxPool`. Channels double at every stage (32→64→128→256→512) while spatial resolution halves. Global average pooling replaces flatten-plus-dense layers, keeping the parameter count low — which matters when training from scratch on only 2,562 images.

Transfer-learning models

The pretrained backbone is loaded with its ImageNet classifier removed, so it outputs a pooled feature vector (1,536 values for B3). A fresh `Dropout → Linear(1536 → 5)` head is attached and the whole network is fine-tuned end to end.

9 Training configuration

Setting	Value	Reason
Optimiser	AdamW	Decoupled weight decay regularises correctly
Learning rate	3e-4	Cosine decay after 2 warmup epochs
Weight decay	1e-5 (transfer) / 1e-4 (scratch)	Stronger for the from-scratch model
Batch size	16 (300 px) / 32 (224 px)	Fits 16 GB unified memory
Loss	Class-weighted cross-entropy, label smoothing 0.05	Counters 49% grade-0 imbalance
Class weights	0.22 / 1.06 / 0.39 / 2.02 / 1.32	Inverse frequency, normalised to mean 1
Gradient clipping	norm 1.0	Prevents exploding updates
Early stopping	patience 8 on validation QWK	Keeps the best checkpoint
Seed	42	Python, NumPy, torch and albumentations all seeded

Why monitor QWK rather than validation loss. Between epochs 10 and 13 the validation *loss* rose while validation *QWK* improved to its best value. Both are correct: cross-entropy punishes overconfidence, whereas QWK only measures whether the predicted grade lands in the right place on the ordinal scale. Early stopping on loss would have frozen the model at epoch 6 and discarded the best result.



EfficientNet-B3 training history. Validation QWK peaks at epoch 13; training halted at epoch 21 under the patience rule.

10 Evaluation metrics

Quadratic Weighted Kappa — the headline number

QWK measures **ordinal agreement** corrected for chance. Unlike accuracy, it weights an error by how far apart the grades are: predicting grade 3 when the truth is 4 costs little, predicting grade 0 when the truth is 4 costs sixteen times as much, because the penalty is the squared distance. 0 means no better than random guessing, 1.0 is perfect. It is the official APTOS competition metric.

QWK is not accuracy. They are different numbers and must not be confused. This model scores **QWK 0.9005** and **accuracy 82.5%**. QWK is higher because nearly every error is an adjacent-grade confusion, which QWK rewards and accuracy does not.

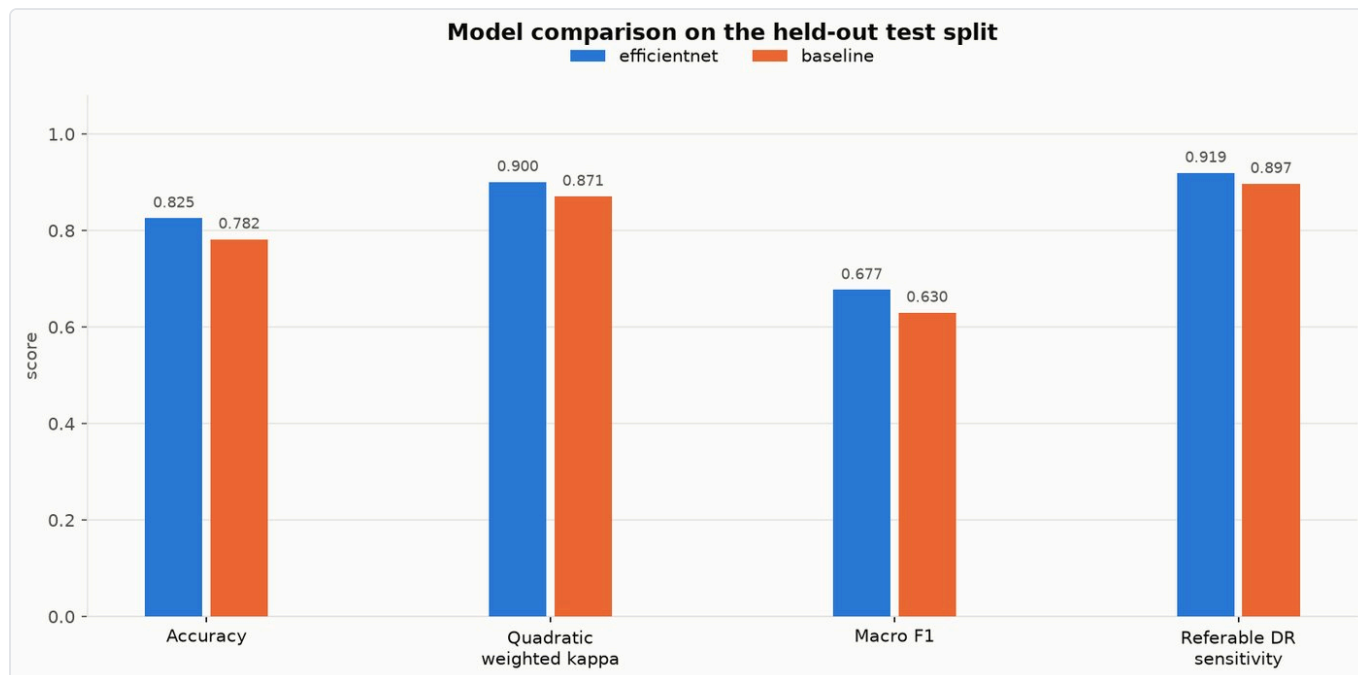
Metric	What it measures	Why it is reported
Quadratic weighted kappa	Ordinal agreement, chance-corrected	Headline metric; matches how a clinician judges errors
Accuracy	Fraction predicted exactly right	Intuitive, but flattered by grade 0 being half the data
Macro F1	Unweighted mean F1 across grades	Treats rare grades as equally important; reveals weakness
Referable sensitivity	Recall on grade ≥ 2	The clinical screening decision — a miss costs eyesight
Referable specificity	True-negative rate on grade ≥ 2	Controls unnecessary referrals

11 Results

Measured on the held-out test split of 550 images, which no training or model-selection step ever touched.

Model	Params	Epochs	Time	Accuracy	QWK	Macro F1	Referable sens.
EfficientNet-B3	10.70 M	21	46 min	0.8255	0.9005	0.6769	0.9193
Baseline CNN	4.85 M	40	22 min	0.7818	0.8706	0.6298	0.8969

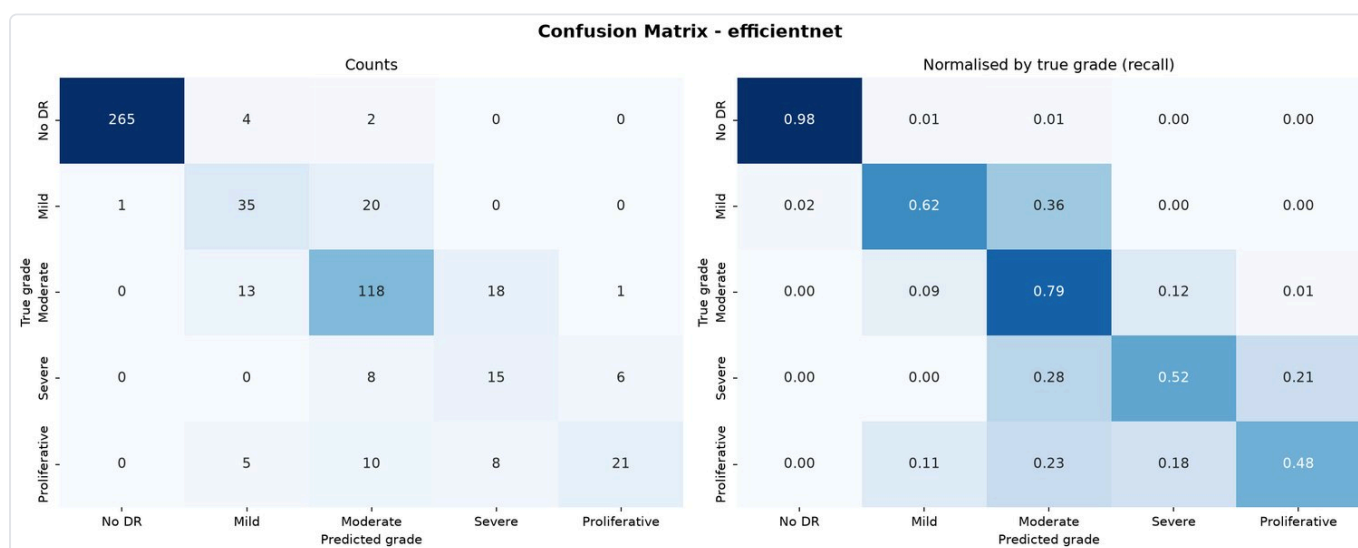
Transfer learning is worth **+0.030 QWK** and **+4.4 percentage points** of accuracy. It also reached the baseline's best score in 4 epochs rather than 32.



Model comparison on the held-out test split.

Per-class performance — EfficientNet-B3

Grade	Precision	Recall	F1	Support
0 - No DR	0.996	0.978	0.987	271
1 - Mild	0.614	0.625	0.619	56
2 - Moderate	0.747	0.787	0.766	150
3 - Severe	0.366	0.517	0.429	29
4 - Proliferative DR	0.750	0.477	0.583	44



Confusion matrix. Counts on the left, normalised by true grade on the right.

Reading the results honestly

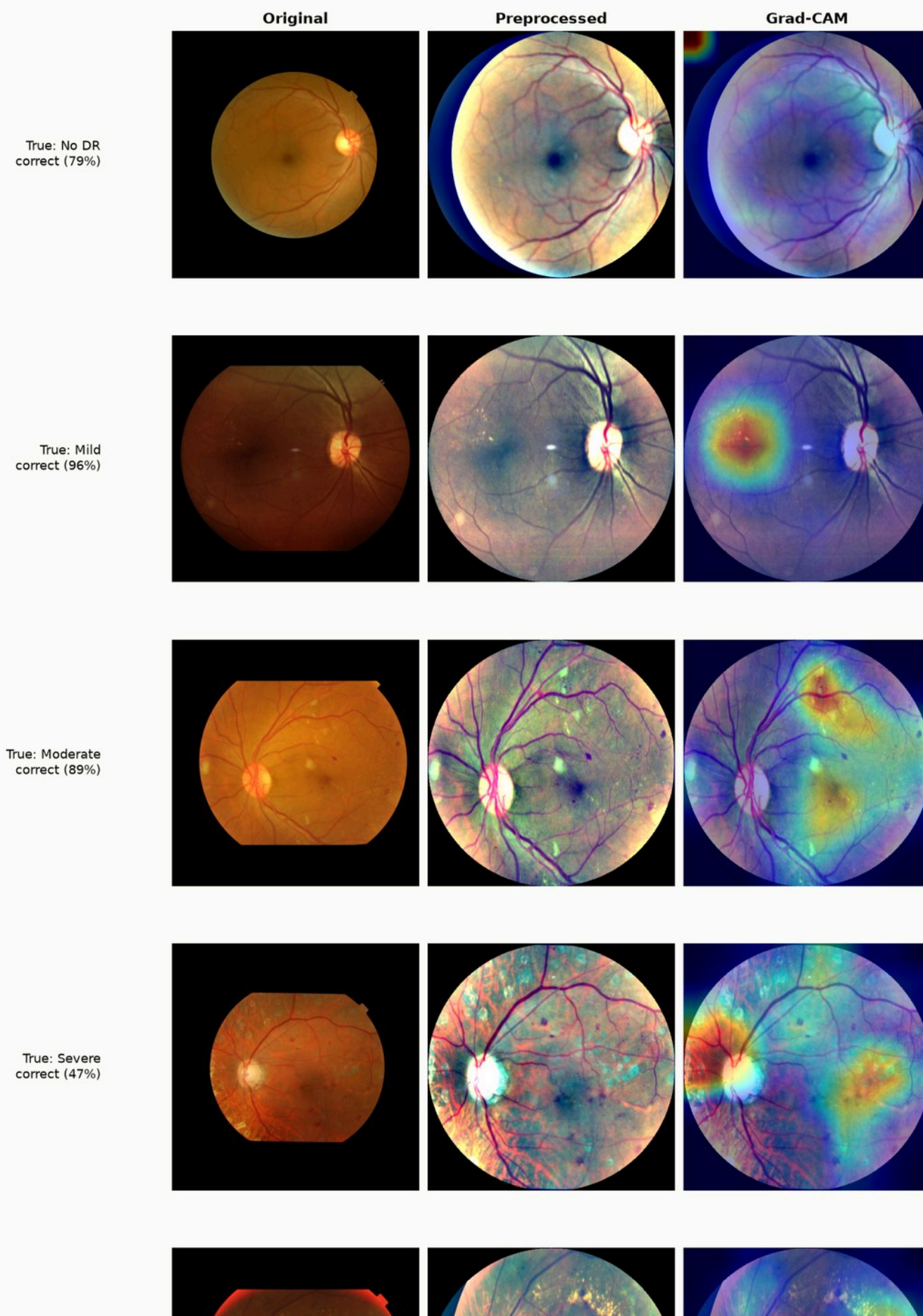
- **No sight-threatening case was called healthy.** Zero grade-3 and grade-4 images were predicted as grade 0 — the bottom-left of the confusion matrix is all zeros. This is the single most important clinical property.
- **Errors are almost all adjacent-grade** (Mild↔Moderate, Severe↔Proliferative), mirroring where human graders themselves disagree.
- **Grades 3 and 4 are the weak point** (F1 0.43 and 0.58). They have only 193 and 295 images in the entire dataset — a data limitation, not a modelling failure.
- **Grade 3 precision is 0.366**, meaning the model over-calls "Severe". That is the deliberate consequence of class weighting: in screening, a false alarm costs an appointment while a miss costs eyesight.
- **Macro F1 (0.68) sits well below accuracy (0.83)** because macro F1 weights all five grades equally while accuracy is flattered by grade 0 being half the data.

12 Explainability

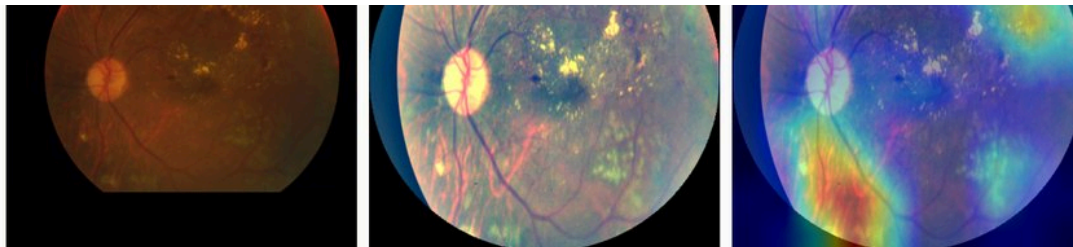
Grad-CAM — where the model looked

Gradient-weighted Class Activation Mapping backpropagates the predicted grade's score to the last convolutional feature map. Channels are weighted by their mean gradient, summed and passed through ReLU, giving a heatmap over the regions that raised the score.

Grad-CAM explanations across DR grades (EfficientNet-B3)



True: Proliferative
correct (95%)



Warmer regions increased the score for the predicted grade.

Grad-CAM across all five grades. On higher grades the heat sits on haemorrhages and exudates; on a healthy retina it is diffuse, because there is no lesion to point at.

Lesion analysis — what is in that region

A heatmap says *where*, not *what*. Classical detectors locate the features the grading scale is actually built on, and the enrichment ratio measures how concentrated each is inside the attended region.

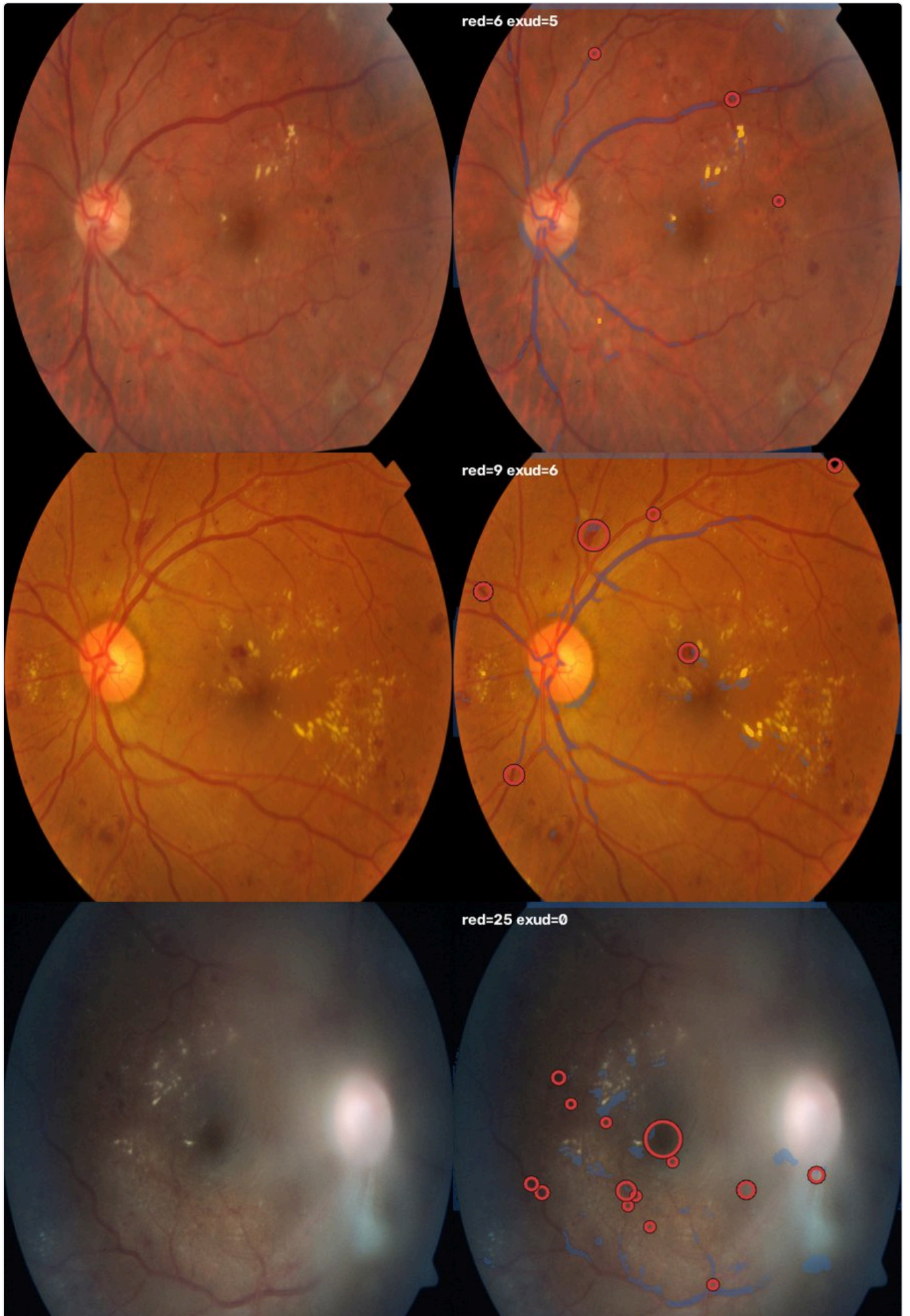
Feature	Detection method	Clinical role
Microaneurysms / haemorrhages	Local minima in the green channel, filtered per blob by area, elongation and fill	Earliest and most decisive sign; number and spread drive grades 1–3
Hard exudates	High brightness and yellowness, optic disc excluded	Lipid deposits from leaking capillaries; sight-threatening near the macula
Vasculature	Black top-hat at multiple orientations	Detected to keep vessels out of the red-lesion mask; drawn for orientation

Detection runs on the *original* image rather than the preprocessed one, because the Ben Graham step recentres every image on mid-grey and destroys the hue difference between yellow exudates and red haemorrhages. The optic disc is masked out of both detectors — it is naturally bright and yellow and would otherwise dominate the exudate count on every image.

Validation across grades

Grade	Exudate count	Red lesion % area	Red lesion count
0	4.5	0.036	6.1
1	5.9	0.090	12.0
2	8.8	0.117	11.6
3	6.9	0.192	16.3
4	9.1	0.213	16.1

Red lesion area rises **5.9×** from grade 0 to grade 4 and is monotonic across all five grades — exactly the criterion the clinical scale uses.



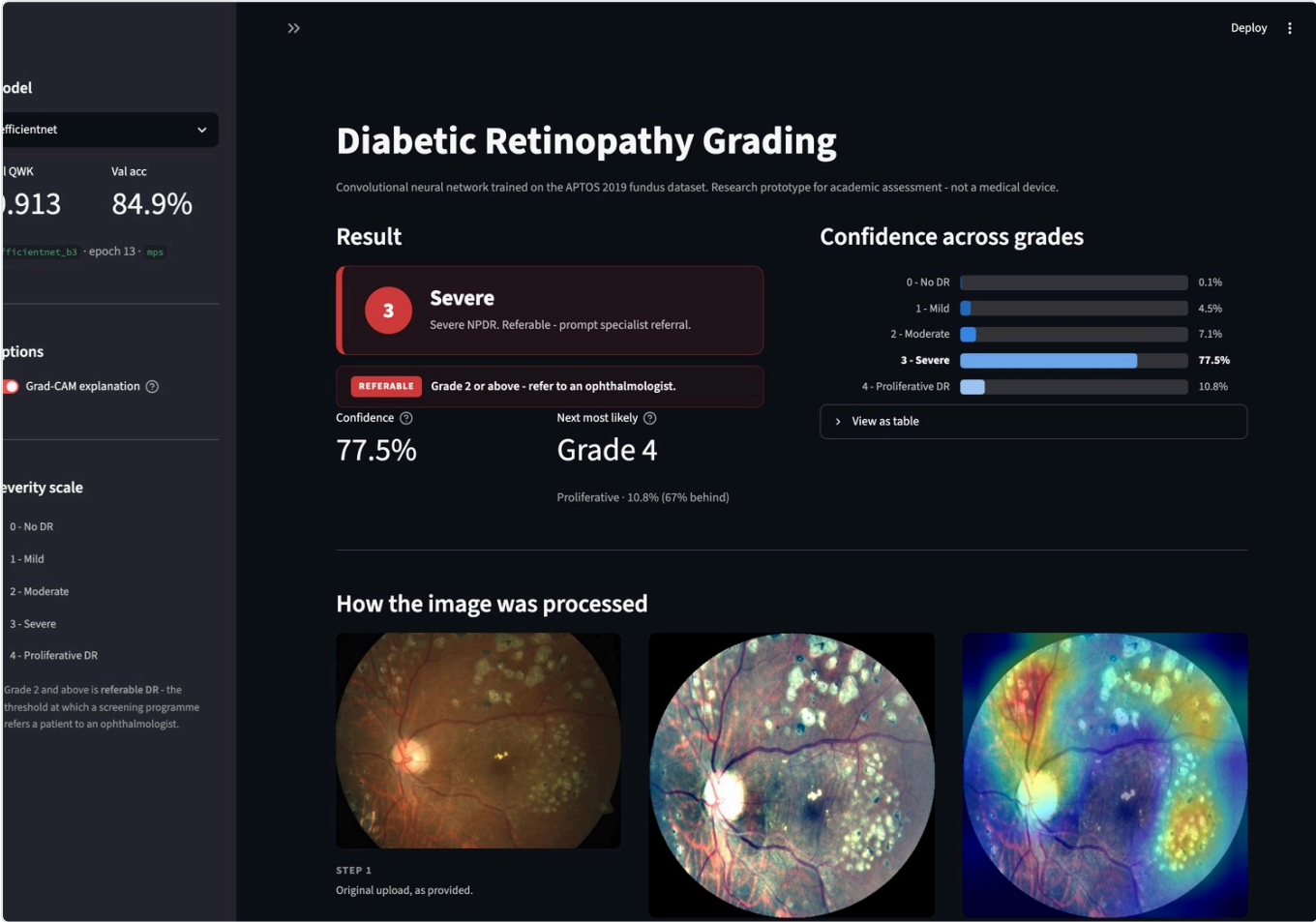


Left: original. Right: detections. Red rings mark microaneurysms and haemorrhages, yellow marks hard exudates, faint blue marks the vasculature.

What this does not prove. APTOS carries image-level grades only, with no lesion annotations, so none of these are trained detectors — they are hand-written image-processing rules. The panel reports a *correlation* between independently detected lesions and where the network looked, not a readout of the model's reasoning. A high multiplier is evidence the model attended to clinically meaningful structures; a low one means either the detectors missed something or the model used features they cannot see.

13 The demo application

A Streamlit application implements stage 7 interactively. Upload a fundus photograph and it returns the grade, per-class confidence, the referable-DR decision, the preprocessing steps, a Grad-CAM overlay and the lesion analysis.



The demo application. The result appears first, above the images, because it is what the user came for.

14 Repository structure

```
src/
  config.py          All hyperparameters, paths and grade definitions
  utils.py           Seeding and reproducibility helpers
  download_data.py   Stage 1 – dataset acquisition and verification
  download_weights.py Fetch trained checkpoints from the GitHub release
  preprocess.py      Stages 2 and 3 – preprocessing and the split
  dataset.py         Dataset class, augmentation, class weights
  models.py          Stages 4–6 – architectures, activations, factory
  engine.py          Training and evaluation loops, metrics, optimisers
  train.py           Training entry point
  evaluate.py        Test-set metrics and figures
  compare.py         Model comparison table and figure
  ablation.py        Optimiser and activation sweeps
  eda.py             Dataset-chapter figures
  gradcam.py         Grad-CAM implementation
  lesions.py         Lesion detection and attention enrichment
  predict.py         Single-image inference
app/app.py           Streamlit demo
notebooks/           Pipeline walkthrough notebook
docs/               Architecture diagram, Windows guide, this document
outputs/figures/     Report figures
outputs/logs/        Metrics, histories, comparison tables
outputs/models/      Trained weights (not committed; see release)
```

15 How to run it

Demo only – no dataset required

```
# macOS / Linux
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
python -m src.download_weights
streamlit run app/app.py

# Windows (PowerShell)
py -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
python -m src.download_weights
streamlit run app/app.py
```

Trained weights are published as GitHub release assets rather than committed, so clones stay small. Full Windows instructions, including the CUDA build and common errors, are in `docs/SETUP_WINDOWS.md`.

Full pipeline — retraining from scratch

```
kaggle auth login                # one-time browser login
python -m src.download_data      # ~8.6 GB
python -m src.preprocess         # cache + stratified split
python -m src.train --model baseline    # ~22 min
python -m src.train --model efficientnet # ~46 min
python -m src.evaluate --model efficientnet --split test
python -m src.compare --split test
python -m src.eda                # dataset figures
```

A two-epoch smoke test verifies the whole pipeline in about a minute:

```
python -m src.train --model efficientnet --epochs 2 --limit 64
```

Ablations

```
python -m src.train --model baseline --optimizer sgd --activation gelu
python -m src.ablation --sweep optimizer --epochs 15
python -m src.ablation --sweep activation --epochs 15
```

Activations available: `relu`, `leaky_relu`, `gelu`, `silu`, `elu`, `mish`. Optimisers: `adamw`, `adam`, `sgd`, `rmsprop`. Sweeps use the baseline model, because pretrained backbones ship with their own activations and replacing those would invalidate the ImageNet weights.

16 Known limitations and future work

Limitation	Detail	How it could be addressed
Test set is a split of APTOS <i>train</i>	The competition's real test labels were never public, so the held-out 550 images come from the labelled training set.	Report it explicitly; it is the standard approach for this dataset.
Rare grades are weak	Grades 3 and 4 have 193 and 295 images in total. F1 is 0.43 and 0.58.	Collect more severe cases, or use focal loss and oversampling.
Label noise	30 duplicate groups are graded differently in different rows.	An irreducible ceiling on achievable accuracy; disclose it.
Grade 3 over-called	Precision 0.366 — a deliberate consequence of class weighting.	Defensible for screening; tune the weights if precision matters more.
Lesion detectors are not trained	APTOS has no lesion annotations, so detection is classical image processing.	Train a U-Net on IDRiD (81 annotated images) or DDR (608).
Microaneurysms are invisible at 300 px	The classifier resizes to 300 px, where a 10-20 px lesion disappears.	Patch-based training at 1024 px or higher.
Single model, no ensemble	Competition-winning solutions ensemble several backbones.	Ensemble EfficientNet, ResNet and a ViT; expect +0.01-0.02 QWK.

17 Glossary

Term	Meaning
Fundus image	A photograph of the interior back surface of the eye, showing retina, optic disc and blood vessels.
Microaneurysm	A tiny bulge in a retinal capillary wall. The earliest visible sign of DR, only 10-20 px in a full-resolution image.
Haemorrhage	Blood leaked into retinal tissue, appearing as a dark red spot.
Hard exudate	A yellow lipid deposit left by leaking capillaries.
Optic disc	The bright circular region where the optic nerve enters the retina. Naturally bright and yellow, so it is masked out of lesion detection.
Macula	The central retinal region responsible for sharp vision. Lesions here are especially sight-threatening.
NPDR / PDR	Non-proliferative and proliferative diabetic retinopathy. PDR involves new, fragile blood vessel growth.
Referable DR	Grade 2 or above — the threshold for referral to an ophthalmologist.
Ordinal classification	Classification where classes have a meaningful order, so some errors are worse than others.
QWK	Quadratic Weighted Kappa. Chance-corrected ordinal agreement, penalising errors by squared distance.
Transfer learning	Reusing a network pretrained on a large dataset (ImageNet) and fine-tuning it on a smaller target task.
Activation function	The non-linearity inside a network (ReLU, SiLU). Without one, stacked layers collapse to a single linear operation.
Optimiser	The rule converting gradients into weight updates (AdamW, SGD).
Class weighting	Scaling the loss per class by inverse frequency so rare classes are not ignored.
Grad-CAM	Gradient-weighted Class Activation Mapping — a heatmap of the regions that raised the predicted class score.
Stratified split	A split preserving each class's proportion across train, validation and test.
Data leakage	When information from the test set influences training — here, duplicate photographs spanning splits.

Disclaimer. Research prototype built for academic assessment. Not a medical device, not clinically validated, and not a substitute for examination by a qualified ophthalmologist.